

评估协议、满窗、成交量，和序列归因 prototype

asof 代码已在 `block_tables.py` / `onepass_post.py`。中间表该落盘（Spark/Scala 同一套 join）。行为序列要做 sequence modeling 时，归因是软的 last-touch 族，没有 GT。prototype：
`scripts/tencent_gr/seq_attr_proto.py`。

1. 两套评估，别混

方法论：你测的是不是你声称的那个量。X 是否只在 `cnv_ts` 已知；Y 是否可观测；切分是否把人的身份漏到测试集；中间表是否可复现。这里可以谈 logloss / AP / 校准——对象必须是可观测 Y（跟满窗后点没点、suffix 里买了几单），不是「哪次点击导致购买」。

业务：成交构成和率。同品漏斗闭了多少、任意路径多少、空路径多少、当场 0 点多少、after vs before。没有归因 label，构成表就是评估。AUC 不能审 last-click 对不对，F-score 更无关。

`cnv_ts` 是锚：买前量、路径、当场都是 `ts ≤ cnv_ts`。买卖的量是另一套 Y：

Y	粒	样本	在问什么
<code>y_post_clk_W</code>	转化	满窗（见下）	这单之后还会不会点
<code>suffix n_cnv / GMV</code>	用户	prefix 切完、suffix 还够长	后面还买不买、买多少
本单 <code>price</code>	转化	全部（成交当下已知）	不预测，是锚上的量

不要拿买后点击的模型去「评估成交量」，也不要吧 `pay_cnt` 当「爱买」。量是次数，结构是 share / 空路径档。

2. 满窗的单 ≠ 评估协议

满窗是样本过滤器，对某个 W：

```
follow = t_end - cnv_ts
正例：窗内已经点到（即使 follow < W）
负例：follow ≥ W 且没点到
follow < W 且没点到 → 丢掉，不当 0
```

评估协议是整张菜谱：粒、Y、X、满窗过滤、按用户切、先构成后模型、消融顺序。满窗只是其中一条。

	满窗过滤？
路径构成（同品/任意/空、SKU 漏斗）	否。backward asof 在 <code>cnv_ts</code> 已完全可观测
<code>dt_item / has_any_clk / sess_clk_before</code>	否
<code>y_post_*</code> 、 <code>n_after</code> 、 <code>lift</code>	是
用户级 <code>post_clk_1d_rate</code>	只平均满窗单
用户 suffix 成交量	不是满窗；是 prefix/suffix 时间够不够

相同点：都是「没看见的未来不当 0」。
不同点：买前构成用全量 12866；买后率只用跟满的那部分（1d 约 12564）。把满窗套到路径占比上，会无故丢掉尾部单，构成会偏。

3. 消融顺序，和按用户切

预报可观测的 `y_post_clk_1d`（不是审归因）时，顺序是把更像热度的先放进去：

```
1 买前任意点击量 n_clk_before_7d      # 热度
2 空路径档 empty_any                    # 这单从哪来
3 dt_any / has_any_clk                  # 任意路径
4 sess_clk_before / sess_pos            # 当场续逛
5 lag_post_clk_1d、lag_empty_any        # 上一单的事后 / 上一单怎么来的
6 同品列                                # 这批方差≈0，放最后当对照
```

先放同品 last-click 是错的：12866 里 14 单有数，消融涨不动说明不了「路径没用」，只说明 **SKU 漏斗这批不存在**。

按用户切：热度和 lag 是人身上的。转化随机切，同一人一单在 train、一单在 test，模型记人，不记「这一单」。shift(1) 防的是本单 Y 进本单 X，防不住身份泄漏。新用户泛化必须 user_id 切。构成表、满窗过滤不依赖这刀；只有「预报」这一段需要。

4. 30 分钟切错，和次数当结构

30min 是续逛的定义，不是要调到分最高。切短了：一场变两场，sess_clk_before=0 变多，续逛低估。切长了：两场粘一场，续逛高估。这批 98.4% 当场 0 点，把 gap 改成 1h 也不太会变成「逛热了再买」——转化本身就常是开场事件。对照报 y_5m / y_1h，不要为了 AUC 改 gap。

次数当结构：n_exp / n_clk / pay_cnt 是量（热度、货架刷了多少）。空路径、同品漏斗是否闭合、cnv_share 是结构。量可以预报量；结构回答「这单从哪来」。别把「点击多所以成交多」写成 SKU 漏斗。

5. SKU 漏斗

```
看见这件 → 点这件 → 买这件
闭合 ⇔ 同品 last-exp 非空 且 (通常) 同品 last-clk 非空
```

任意 last-click 闭合的是「人在场」，不是这件漏斗。

这批：同品点过 0.11%，同品曝光过 0.20%。漏斗几乎全开。特征仍要算：它是**监控列**——哪天日志把成交 SKU 对上曝光了，这 0.11% 会跳，主路径才切回 SKU。

6. 这 12866 单上：热度、泄漏、连单、删失、曝/点、滞后、空路径滞后、同品监控

数字都来自同一 prefix。

热度。买前任意点击量预报买后任意点击（volume 就能拉开）。空路径那半买后 1d 只有 6.8%，有任意点击的 15.7%——空路径和冷用户叠在一起。分层：先按 n_clk_before_7d 切热/冷，再看空路径的买后率，才知道是「没点过这件」还是「这人根本不在 seq 里逛」。

泄漏。本单 n_after/y 进 X；lag 不 shift；跟不满当 0；按转化随机切用户。满窗规则防的是第三种。

连单。任意 n_after 会吃到下一单买前的点。1d after≈before 已经够脏；同品 after 干净但基数≈0。要看「买完还点」用 next 且 dt>30min，或只计下场。

右删失。1d 丢掉约 300 单（12866→12564）。不当 0。路径构成仍用 12866。

左切。prefix 0.75 会切掉更早点，假空路径。所以空路径 = 「这段 prefix 里看不见点」，不是「终身没点过」。

曝光量 vs 点击量。这批事件 39.7 万曝 / 1.3 万点 / 1.3 万转化。点≈转化，但同品对不上：点的不是买的那件。曝量是货架热度（任意 n_exp_before）；点击量是动手热度；都不是 SKU 漏斗。同品曝量≈0 先当**覆盖**，不当漏点。

滞后（直观）。lag_post_clk_1d = 此前满窗单的买后点击率，shift(1)。人买完爱不爱点。短窗续逛不该靠它；1d/7d 平台回访会靠它。

空路径当滞后。空路径是**这一单怎么来的**（empty_any=1{prefix 里没有任何点击}）。滞后是**上一单的事后**。可以再**做一列人身上的来路**：

```
lag_empty_any = mean( 此前各单的 empty_any )
```

和 lag_post_clk 并排：这个人是不是经常「seq 外成交」，以及那种人买完还点不点。第一单两列都空。不要把 empty_any 填进 dt_any=0，也不要 lag_empty 代替 lag_post_clk。

同品列留监控。 每周（或每次跑表）看：同品 last-clk 对上占比、同品 last-exp 占比、y_post_same_1d 基数。现在 ~0.1%，主模型不用。一旦跳起来（对上曝光、商详流量进来），同品 dt_item / within_5m 才进主信号。监控是构成表，不是把常数列塞进模型「备着」。

7. 中间表要存（Spark 同一套）

不要每次从 seq 重扫。落盘：

ev	user × event	
attr	user × cnv	backward asof (曝/点 × 同品/任意)
post	user × cnv	forward asof + 满窗 y + n_before/after
sess	user × 场	或把 sess_* 直接写在 attr/post 上
user	user × 1	未删失聚合后再 join

scripts/tencent_gr/dump_block_tables.py → results/tencent_gr_fs150/tables/*.parquet

Spark/Scala: pandas merge_asof = 按键分区、按时间排序的 last/first。3.3+ 可用 ASOF JOIN，否则窗口：

```
W = Window.partitionBy("user_id","item_id").orderBy("ts")
clk.withColumn("cum", count("*").over(W))
# last clk before cnv:
last_clk = clk.withColumn("clk_ts", col("ts"))
cnv.join(last_clk, (cnv.user===clk.user) && (cnv.item===clk.item) && (clk.ts<=cnv.ts))
    .withColumn("rn", row_number().over(Window.partitionBy(cnv.keys).orderBy(clk.ts.desc)))
    .where(col("rn")==1)
```

forward: clk.ts > cnv.ts 取 min。累计差分还是 C(t)-C(t-W)。满窗、shift(1) 的 lag 都是窗口函数，和这份 pandas 表一一对应。

8. 行为序列 → sequence modeling：业务是什么

表模型：把路径收成几个标量（last dt、n_before）。

序列模型：转化前的 (item, act, ts, price) **整段留下**，让模型自己加权。业务上这是 **软归因**：把这一单的成交（或 GMV=price）分到前面的曝光/点击上。

仍无 GT。last-touch / 均匀 / 时间衰减是规则归因；attention 是可学习的同一族。训练代理只能是可观测的：下一步点什么、这段历史后会不会买、买后满窗会不会再点。注意力权重要当「分账」，不要当「因果」。

同品 mask vs 任意 mask 还是那两问：权重在同品触点 = SKU 漏斗；在任意点击 = 人在场。这批同品触点几乎没有，序列模型若学成 last-any-click，和构成表一致。

价格：query 侧用本单 log1p(price)，key 侧用触点当时价格（有的话）。GMV 分账：credit_k = α_k * price_cnv。

prototype 实现了：规则 last-touch（同品点 / 任意点 / 同品曝）+ 时间衰减 + 一个显式权重的 softmax 分账。见下节代码。